

Thibaut Waechter
Promotion : 2024-2025
Année universitaire : 2024-2025

Tutoriel d'utilisation des drones Crazyflies



Organisme d'accueil : iCube
300 Boulevard Sébastien Brandt,
67400 Illkirch



Sylvain Durand
sylvain.durand@insa-strasbourg.fr
03/03/25 - 05/09/25

Table des matières

1	Installation de ROS2	2
1.1	Configuration du système	2
1.2	Ajout des dépôts nécessaires et installation des prérequis	2
1.3	Installation de ROS2	3
2	Installation de cfclient	4
2.1	Installation de pip et Python	4
2.2	Installation de cfclient	4
2.3	Première utilisation de <i>cfclient</i>	4
3	Crazyflie avec ROS2	6
3.1	Installation	6
3.1.1	Installation de Gazebo Ionic	6
3.1.2	Installation de la bibliothèque CrazyFlie	6
3.1.3	Première manipulation de Gazebo Ionic	7
3.1.4	Lien avec un CrazyFlie réel	9
4	Package eSWARM	10
4.1	Package <i>controller</i>	10
4.1.1	Téléchargement du package	10
4.1.2	<i>dualshock_test</i>	10
4.1.3	<i>teleop_dualshock</i>	10
4.1.4	<i>attitude_dualshock</i>	10
4.2	Package <i>control</i>	11
4.2.1	Téléchargement du package	11
4.2.2	<i>test_control.launch.py</i>	11
4.2.3	<i>vel_control.launch.py</i>	11
4.2.4	<i>att_control.launch.py</i>	11
4.3	Package <i>simu</i>	12
4.3.1	Téléchargement du package	12
4.3.2	<i>simu_launch.py</i>	12
A	Installation de Ubuntu sous VM	15
A.1	Prérequis	15
A.2	Création de la machine virtuelle	15
A.3	Configuration de la machine virtuelle	17
A.4	Configuration du clavier et de la résolution	17
B	Installation de Visual Studio Code	18

1 Installation de ROS2

Il est possible de retrouver ce tutoriel en cliquant [ici](#). Il est conseillé de suivre le tutoriel sur le site internet plutôt que sur le présent document car il est plus simple de *copier-coller* les commandes (les retours à la ligne, symbolisés par des flèches sur ce document, posent problème).

1.1 Configuration du système

Avant d'installer ROS2, il est nécessaire de mettre à jour le système. Suivre les étapes suivantes :

1. Mettre à jour les paquets et installer les locales :

```
sudo apt update && sudo apt install locales
```

2. Générer les locales pour l'anglais des États-Unis (en_US) :

```
sudo locale-gen en_US en_US.UTF-8  
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
```

3. Exporter la variable d'environnement pour la langue :

```
export LANG=en_US.UTF-8
```

4. Vérifier les paramètres des locales :

```
locale
```

1.2 Ajout des dépôts nécessaires et installation des prérequis

1. Installer les outils nécessaires pour gérer les dépôts logiciels :

```
sudo apt install software-properties-common
```

2. Ajouter le dépôt "universe" :

```
sudo add-apt-repository universe
```

3. Mettre à jour les paquets et installer curl :

```
sudo apt update && sudo apt install curl -y
```

4. Télécharger la clé ROS2 :

```
sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/  
→ master/ros.key -o /usr/share/keyrings/ros-archive-keyring.gpg
```

5. Ajouter le dépôt ROS2 à la liste des sources :

```
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/  
→ keyrings/ros-archive-keyring.gpg] http://packages.ros.org/ros2  
→ /ubuntu $(. /etc/os-release && echo \jammy) main" | sudo tee /  
→ etc/apt/sources.list.d/ros2.list > /dev/null
```

1.3 Installation de ROS2

1. Mettre à jour et améliorer le système :

```
sudo apt update  
sudo apt upgrade
```

2. Installer les paquets de base de ROS2. Deux options sont disponibles selon l'utilisation prévue de ROS2 :

- **Installation Desktop (Recommandée)** : Inclut ROS, RViz, des démonstrations et des tutoriels.

```
sudo apt install ros-humble-desktop
```

- **Installation ROS-Base (Version minimale)** : Inclut les bibliothèques de communication, les paquets de messages et les outils en ligne de commande.

```
sudo apt install ros-humble-ros-base
```

3. Après avoir choisi entre ROS-Base et ROS-Desktop, ajouter l'environnement ROS2 au chemin système :

```
echo source /opt/ros/humble/setup.bash >> ~/.bashrc  
source ~/.bashrc
```

2 Installation de cfclient

Le tutoriel provient du site suivant :

www.bitcraze.io/documentation/tutorials/getting-started-with-crazyflie-2-x.

2.1 Installation de pip et Python

```
sudo apt install python3-pip  
pip3 install --upgrade pip
```

2.2 Installation de cfclient

Il faut tout d'abord installer *git* :

```
sudo apt install git
```

Puis :

```
git clone https://github.com/bitcraze/crazyflie-clients-python  
cd crazyflie-clients-python
```

Il est alors possible d'installer *cfclient* :

```
pip3 install -e
```

Avant de lancer le logiciel, il faut ajouter une ligne dans le *bashrc* :

```
cd ~/  
echo 'export PATH=$PATH:~/.local/bin' >> ~/.bashrc  
source ~/.bashrc
```

Si tout s'est bien passé, le logiciel peut se lancer avec la commande suivante :

```
cfclient
```

2.3 Première utilisation de *cfclient*

Lancer *cfclient* en utilisant la commande suivante dans un terminal :

```
cfclient
```

La fenêtre suivante devrait alors s'ouvrir :

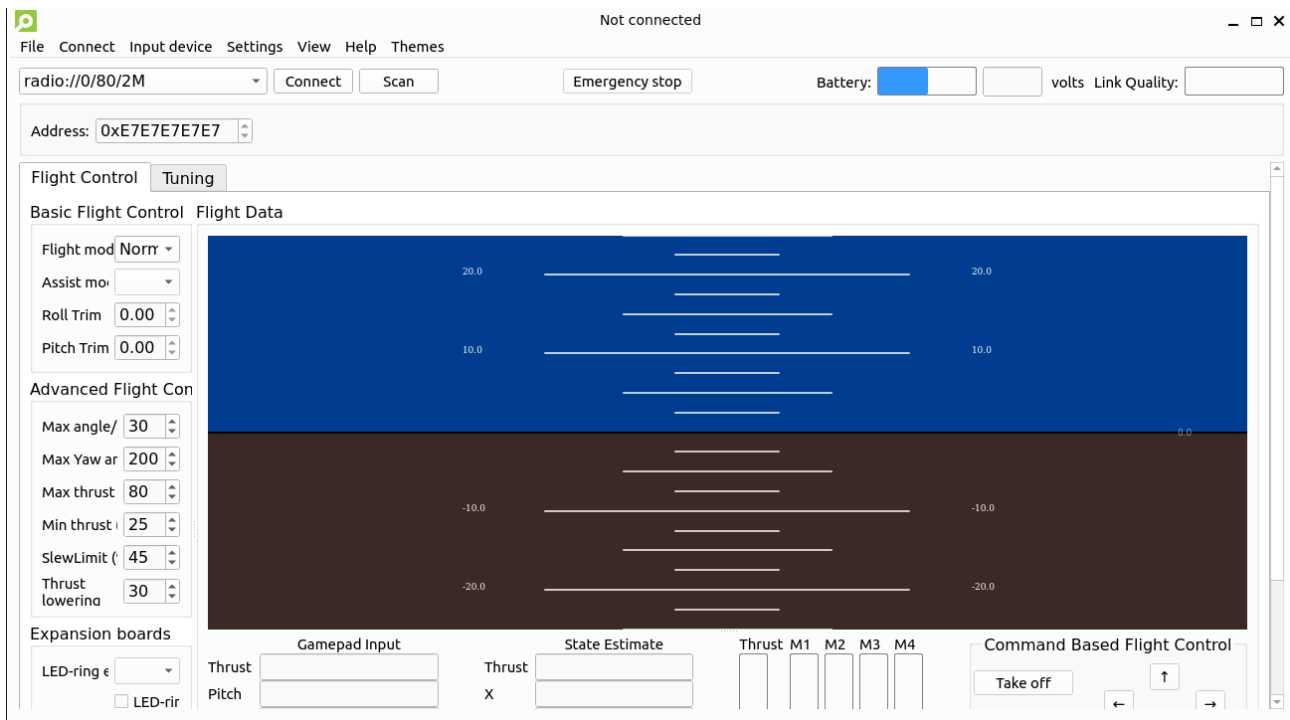


FIGURE 2.1 – cfclient

Connecter le Crazyradio au PC afin de permettre la détection du Crazyflie. Si celui-ci n'a jamais été utilisé, son adresse radio par défaut est 0xE7E7E7E7E7 (voir en haut à gauche sur cfclient). Saisir l'adresse correcte, cliquer sur *Scan*, puis, une fois le drone détecté, cliquer sur *Connecter*.

Pour mettre à jour le firmware du Crazyflie :

1. Cliquer sur *Connect*, puis sélectionner *Bootloader*.
2. Si l'appareil est un Crazyflie 2.1 classique, choisir *cf2*, sélectionner le firmware souhaité, puis cliquer sur *Programmer*.
3. Patienter jusqu'à la fin de la mise à jour. Le Crazyflie peut redémarrer plusieurs fois au cours du processus.

Conformément aux conventions de *Crazyswarm*, si ce Crazyflie doit être le premier de l'essaim, son adresse doit être 0xE7E7E7E701. Pour modifier cette adresse :

1. Aller dans *Connecter*, puis *Configurer 2.X*.
2. Saisir l'adresse souhaitée et cliquer sur *Écrire*.
3. Redémarrer le drone pour appliquer la modification.

Cette application permet également d'autres fonctionnalités, telles que le réglage des gains PID ou le contrôle du Crazyflie avec une manette. Le guide utilisateur est disponible [ici](#).

3 Crazyflie avec ROS2

3.1 Installation

3.1.1 Installation de Gazebo Ionic

Après avoir installé ROS2, il faut ensuite télécharger Gazebo Ionic, un logiciel de simulation. Pour cela, un tutoriel est disponible ici :

https://gazebo-sim.org/docs/latest/install_ubuntu/

Les commandes suivantes proviennent directement de ce tutoriel.

Il faut dans un premier temps télécharger des bibliothèques nécessaires à Gazebo Ionic :

```
sudo apt-get update
sudo apt-get install lsb-release gnupg
```

Il est ensuite possible d'installer Gazebo Ionic :

```
sudo curl https://packages.osrfoundation.org/gazebo.gpg --output /usr/
↪ share/keyrings/pkgs-osrf-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/
↪ keyrings/pkgs-osrf-archive-keyring.gpg] http://packages.
↪ osrfoundation.org/gazebo/ubuntu-stable $(lsb_release -cs) main" |
↪ sudo tee /etc/apt/sources.list.d/gazebo-stable.list > /dev/null
sudo apt-get update
sudo apt-get install gz-ionic
```

Note : Il est encore une fois conseillé de copier les commandes depuis le site internet source, afin d'éviter les erreurs possibles en copiant depuis le présent pdf. Les flèches sur la commande ci-dessus représente un saut à la ligne, ce qui ne doit pas être présent dans la commande à rentrer dans le terminal.

3.1.2 Installation de la bibliothèque CrazyFlie

Le tutoriel est disponible ici :

<https://www.bitcraze.io/2024/09/crazyflies-adventures-with-ros-2-and-gazebo/>

Cette section décrit l'installation de quatre paquets :

- **crazyflie_simulation** : Contient le modèle Gazebo du Crazyflie avec un lidar en tant que multiranger.
- **crazyswarm2** : Fournit le lien de communication et l'intégration ROS2 avec le Crazyflie.
- **ros_gz_crazyflie** : Gère le pont entre Gazebo et ROS2.
- **crazyflie_ros2_multiranger** : Gère la cartographie simple et l'autonomie de suivi de mur.

Créer les espaces de travail pour la simulation et ROS :

```
mkdir ~/crazyflie_mapping_demo
cd crazyflie_mapping_demo
mkdir simulation_ws
```

```
mkdir ros2_ws
cd ros2_ws
mkdir src
```

Cloner les dépôts dans les répertoires appropriés, en commençant par la simulation :

```
cd ~/crazyflie_mapping_demo/simulation_ws
git clone https://github.com/bitcraze/crazyflie-simulation.git
```

Ensuite, naviguer vers le dossier `src` de l'espace de travail ROS2 et cloner trois projets :

```
cd ~/crazyflie_mapping_demo/ros2_ws/src
git clone https://github.com/knmcguire/crazyflie_ros2_multiranger.git
git clone https://github.com/knmcguire/ros_gz_crazyflie
git clone --recursive https://github.com/IMRCLab/crazyswarm2
```

Installer les bibliothèques nécessaires via `apt-get` et `pip` (il est recommandé d'utiliser un environnement virtuel Python) :

```
sudo apt-get install libboost-program-options-dev libusb-1.0-0-dev
    ↪ python3-colcon-common-extensions
sudo apt-get install ros-humble-motion-capture-tracking ros-humble-tf-
    ↪ transformations
sudo apt-get install ros-humble-ros-gzharmonic ros-humble-teleop-twist-
    ↪ keyboard
pip3 install cflib transform3D
```

Se rendre dans l'espace de travail ROS2 et compiler les paquets :

```
cd ~/crazyflie_mapping_demo/ros2_ws/
source /opt/ros/humble/setup.bash
colcon build --cmake-args -DBUILD_TESTING=ON
```

La compilation prendra quelques minutes. En particulier, `Crazyswarm2` affichera de nombreux avertissements et erreurs standard (**stderr**), mais tant que la compilation ne se termine pas par un échec, ces messages sont à ignorer.

3.1.3 Première manipulation de Gazebo Ionic

Cette section explique comment utiliser les packages téléchargé précédemment pour simuler un CrazyFlie sur Gazebo à travers un exemple simple.

Ouvrir un terminal et s'assurer qu'il est bien configuré pour utiliser à la fois le modèle Gazebo et les paquets ROS2 nouvellement compilés :

```
source ~/crazyflie_mapping_demo/ros2_ws/install/setup.bash
export GZ_SIM_RESOURCE_PATH="/home/$USER/crazyflie_mapping_demo/
    ↪ simulation_ws/crazyflie-simulation/simulator_files/gazebo/"
```

Démarrer la simulation en lançant les programmes ROS2 :


```
ros2 launch crazyflie_ros2_multiranger_bringup simple_mapper_simulation.  
→ launch.py
```

Si une erreur "No such file or directory" apparaît concernant le modèle, essayer de spécifier le chemin complet dans l'exportation de `GZ_SIM_RESOURCE_PATH`.

Gazebo démarrera avec le Crazyflie au centre de la scène. Pour avoir une vue rapprochée du Crazyflie, effectuer un clic droit sur celui-ci dans l'arborescence des entités et sélectionner `Move to`.

Ouvrir un autre terminal, sourcer ROS2 et exécuter le nœud `teleop_key` via le clavier :

```
source /opt/ros/humble/setup.bash  
ros2 run teleop_twist_keyboard teleop_twist_keyboard
```

Cela permet de piloter le CrazyFlie virtuel avec un clavier.

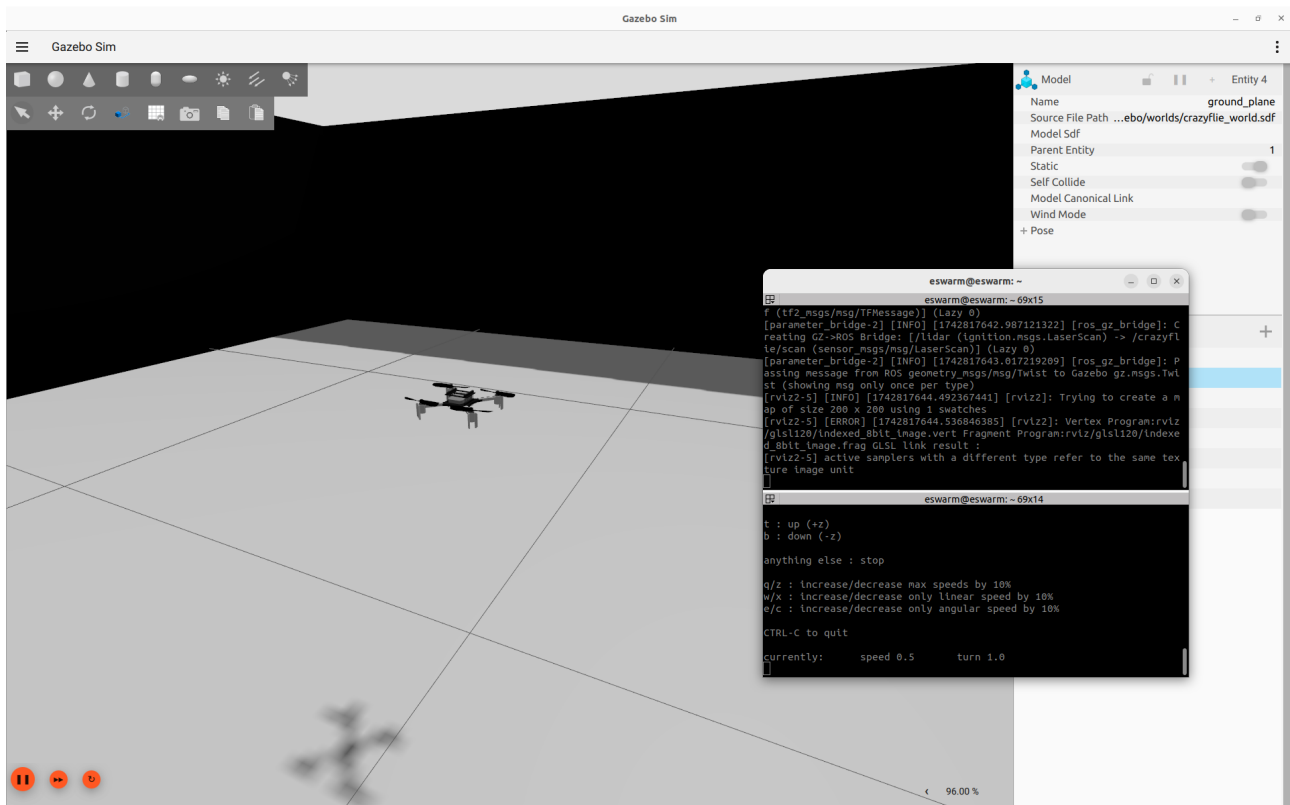


FIGURE 3.1 – Crazyflie sur Gazebo

Le drone de la simulation est piloté en vitesse linéaire et angulaire, en suivant les commandes envoyées sur le topic `cmd_vel`. Au lieu d'utiliser le programme `teleop_twist_keyboard`, il est évidemment d'utiliser un autre nœud, comme par exemple `teleop_dualshock.py` (voir annexe 4.1.3), pour utiliser une manette.

3.1.4 Lien avec un CrazyFlie réel

Il est conseillé d'installer Visual Studio Code, ou tout autre IDE, afin de faciliter la modification de plusieurs fichiers. Le tutoriel d'installation de Visual Studio Code pour Ubuntu est donné en annexe B. Ouvrir ensuite le dossier *crazyflie_mapping_demo* dans l'IDE, et ouvrir le fichier suivant dans un éditeur de texte :

```
~/crazyflie_mapping_demo/ros2_ws/src/crazyflie_ros2_multiranger/  
  ↳ crazyflie_ros2_multiranger_bringup/config/  
  ↳ crazyflie_real_crazyswarm2.yaml
```

Et s'assurer que la ligne 5, comme montré ci-dessous, correspond bien à l'URL radio configuré précédemment.

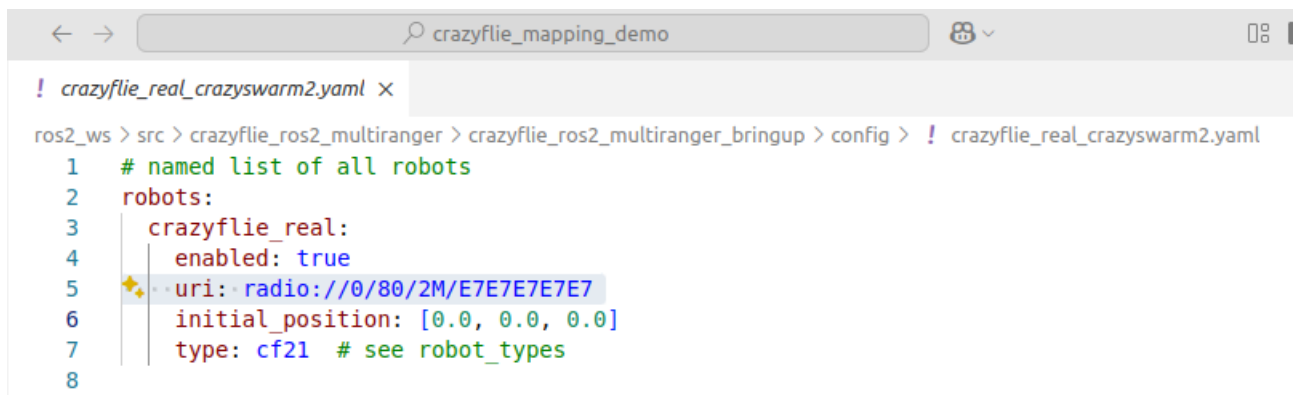


FIGURE 3.2 – Configuration radio

Puis, lancer le programme :

```
ros2 launch crazyflie_ros2_multiranger_bringup simple_mapper_real.launch  
  ↳ .py
```

Comme pour la simulation précédente, le drone est piloté en suivant les commandes de vitesses linéaires et angulaires du topic *cmd_vel*. Il faut donc à nouveau lancer un noeud de pour envoyer ces commandes, comme le *teleop_key* ou *teleop_dualshock* :

```
ros2 run teleop_twist_keyboard teleop_twist_keyboard
```

4 Package eSWARM

4.1 Package *controller*

4.1.1 *Téléchargement du package*

Se placer dans le dossier de travail avec `cd` :

```
cd ~/crazyflie_mapping_demo/ros2_ws/
```

Et cloner le Github suivant :

```
git clone git@github.com:watson67/controller_repo.git
```

Puis, *build* le dossier :

```
source /opt/ros/humble/setup.bash  
colcon build --cmake-args -DBUILD_TESTING=ON
```

Note : Ce package a été réalisé en utilisant une manette Dualshock4 (manette de PlayStation 4) et n'a pas été testé avec d'autres types de manettes. Si un autre type est utilisé, lance le programme `dualshock_test` afin de tester les entrées de la manette utilisée.

4.1.2 *dualshock_test*

```
ros2 run controller dualshock_test
```

Ce programme permet de tester une manette en affichant le nom des boutons pressés, ainsi que la valeur des sticks analogiques.

4.1.3 *teleop_dualshock*

```
ros2 run controller teleop_dualshock
```

Ce programme permet d'envoyer des commandes de vitesses linéaires et angulaires sur le topic `cmd_vel`

4.1.4 *attitude_dualshock*

```
ros2 run controller attitude_dualshock
```

Ce programme permet d'envoyer des commandes de roll/pitch/yaw/thrust sur le topic `cmd_vel_legacy`.

4.2 Package control

Ce package contient différents programmes simples permettant de piloter un drone CrazyFlie. Les programmes utilisés sont des versions simplifiés de ceux utilisés dans la partie 3.1.4.

4.2.1 Téléchargement du package

Se placer dans le dossier de travail avec `cd` :

```
cd ~/crazyflie_mapping_demo/ros2_ws/
```

Et cloner le Github suivant :

```
git clone git@github.com:watson67/control_repo.git
```

Puis, *build* le dossier :

```
source /opt/ros/humble/setup.bash  
colcon build --cmake-args -DBUILD_TESTING=ON
```

4.2.2 *test_control.launch.py*

Rappel : Un fichier *launch* permet de lancer plusieurs noeuds ROS simultanément.

```
ros2 launch control test_control.launch.py
```

L'objectif de ce programme est de faire décoller le drone, puis le faire atterrir, par pression de la barre espace. Pour cela, il faut lancer le programme *teleop.py* dans un second terminal.

```
ros2 run control teleop.py
```

Ce programme permet d'envoyer les commandes de lancement du décollage ou de l'atterrissage par pression de la barre espace. Appuyer sur *e* lance l'arrêt d'urgence.

4.2.3 *vel_control.launch.py*

```
ros2 launch control vel_control.launch.py
```

Ce fichier permet de piloter un drone en lui envoyant des vitesses linéaires et angulaires sur le topic *cmd_vel*. Lancer le noeud *teleop_dualshock* dans un second terminal pour piloter le drone à la manette.

4.2.4 *att_control.launch.py*

Non fonctionnel

```
ros2 launch control att_control.launch.py
```

Ce fichier permet de piloter un drone en lui envoyant des commandes roll/pitch/yaw/thrust sur le topic `cmd_vel_legacy`. Lancer le noeud `attitude_dualshock` dans un second terminal pour piloter le drone à la manette.

4.3 Package *simu*

Ce package a pour objectif de simuler deux Crazyflies.

4.3.1 Téléchargement du package

Se placer dans le dossier de travail avec `cd` :

```
cd ~/crazyflie_mapping_demo/ros2_ws/
```

Et cloner le Github suivant :

```
git clone git@github.com:watson67/simulation_eswarm.git
```

4.3.2 *simu_launch.py*

Cette commande permet de lancer la simulation Gazebo :

```
ros2 launch simu simu_launch.py
```

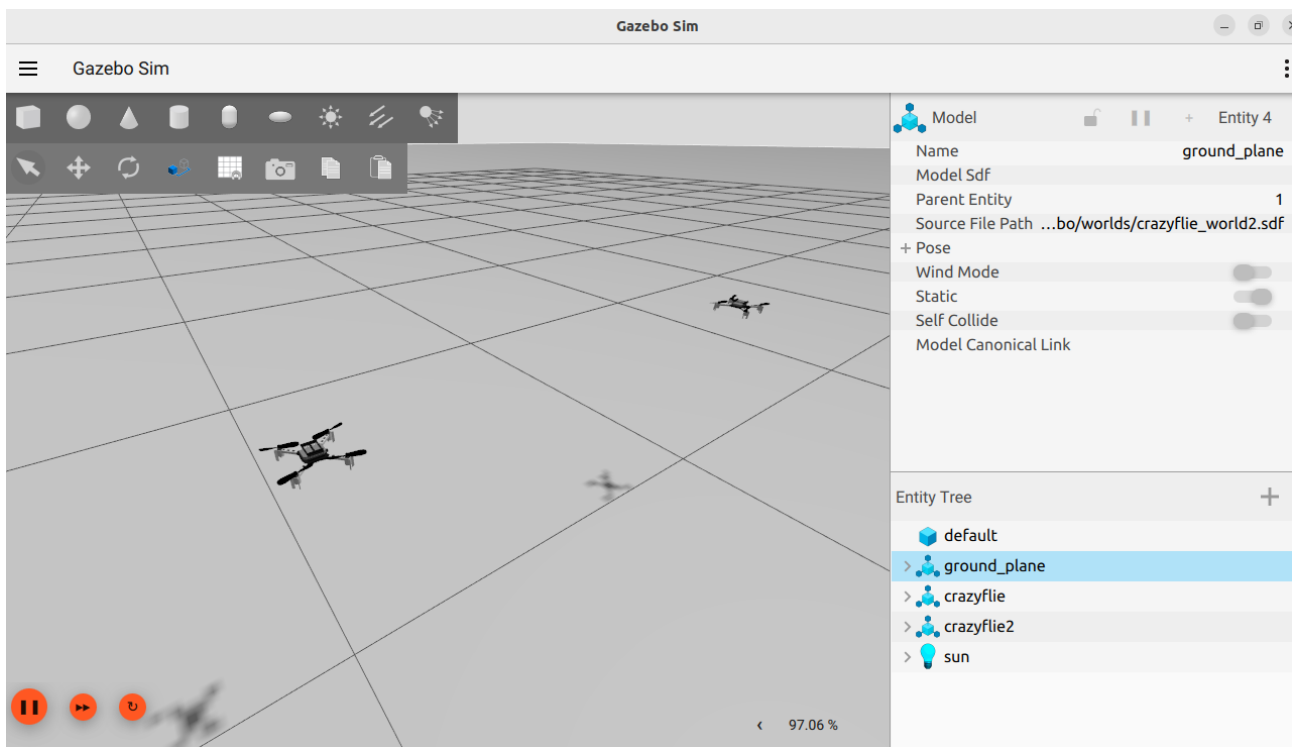


FIGURE 4.1 – Simulation de deux Crazyflies sur Gazebo

Pour l'instant, ces deux drones sont pilotés par la même commande. Il n'est pas possible de piloter chaque Crazyflies indépendamment. De plus, ils sont ocmmandés en vitesse linéaire et angulaire.

Pour les piloter, utiliser le *teleop_twist* :

```
ros2 run teleop_twist_keyboard teleop_twist_keyboard
```

Ou une manette :

```
ros2 run controller teleop_dualshock
```

Table des figures

2.1	cfclient	5
3.1	Crazyflie sur Gazebo	8
3.2	Configuration radio	9
4.1	Simulation de deux Crazyflies sur Gazebo	12
A.1	Initialisation de la machine virtuelle	15
A.2	Choix de la RAM pour la VM	16
A.3	Choix de l'espace disque pour la VM	16
A.4	Accélération graphique	17

A Installation de Ubuntu sous VM

Un tutoriel d'installation de Ubuntu 20.04 et ROS2 est disponible en cliquant [ici](#). L'installation sur machine virtuelle suit des étapes similaires, le tutoriel ci-dessous reprend les informations de ce site internet.

Note : Si l'objectif final est de faire de la simulation, il n'est pas recommandé d'utiliser une machine virtuelle, les performances sous VM ne permettent pas de faire tourner Gazebo convenablement.

A.1 Prérequis

- Télécharger l'image ISO de Ubuntu 22.04 depuis le site officiel de Ubuntu (desktop) : <https://releases.ubuntu.com/jammy/>
- Télécharger et installer Oracle VM VirtualBox depuis le site officiel de VirtualBox : <https://www.virtualbox.org/wiki/Downloads>

A.2 Création de la machine virtuelle

- Ouvrir VirtualBox.
- Cliquer sur "Nouvelle" pour créer une nouvelle machine virtuelle.
- Donner un nom à la machine virtuelle (par exemple "Ubuntu 20.04") et sélectionner le type "Linux" et la version "Ubuntu (64 bits)".
- Charger l'image ISO d'Ubuntu 22.04 téléchargée précédemment.
- Cliquer sur "Skip Unattended Installation".

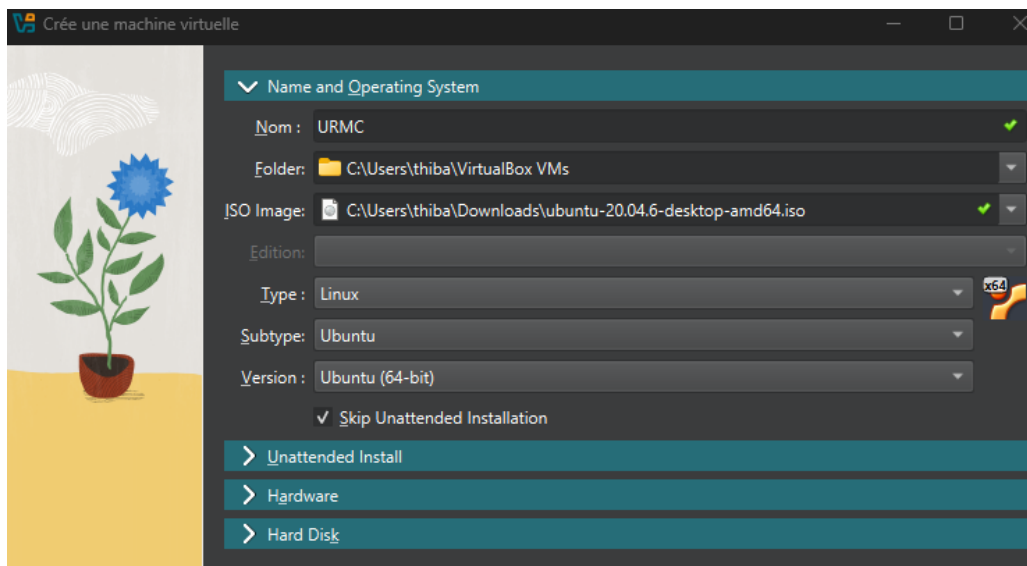


FIGURE A.1 – Initialisation de la machine virtuelle

- Créer un domaine d'utilisateur, avec un nom d'utilisateur (Username) et un mot de passe (Password).

- Définir la taille de la mémoire vive (RAM) allouée à la machine virtuelle (un minimum de 4 Go est recommandé).

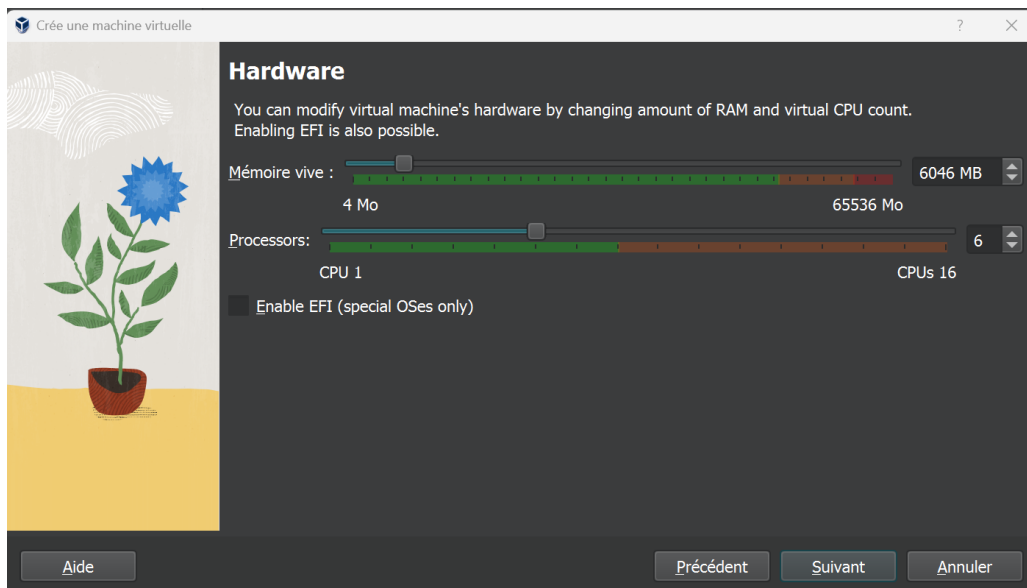


FIGURE A.2 – Choix de la RAM pour la VM

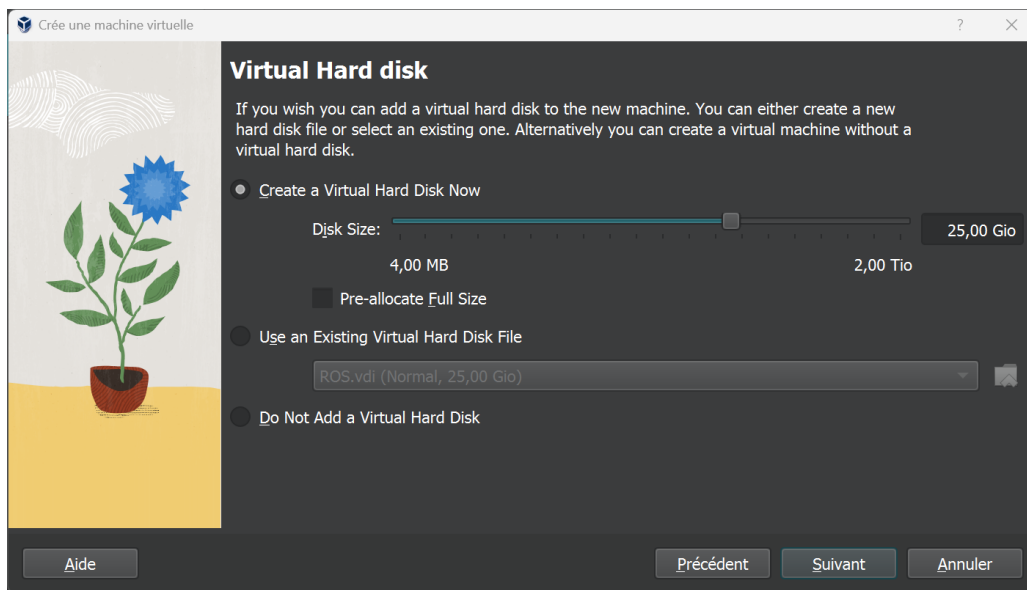


FIGURE A.3 – Choix de l'espace disque pour la VM

- Cliquer sur "Créer" pour finaliser la création de la machine virtuelle.

Avant de lancer la machine virtuelle, configurer quelques paramètres supplémentaires : Sélectionner la machine virtuelle, appuyer sur "Configuration" et aller dans l'onglet "Expert" en haut à gauche, puis "Affichage". Activer, si l'ordinateur le permet, l'accélération graphique et augmenter la mémoire vidéo. Cela améliore les performances graphiques de la machine virtuelle. Cependant, selon la version de Gazebo utilisée, cela peut complètement empêcher le logiciel de fonctionner.

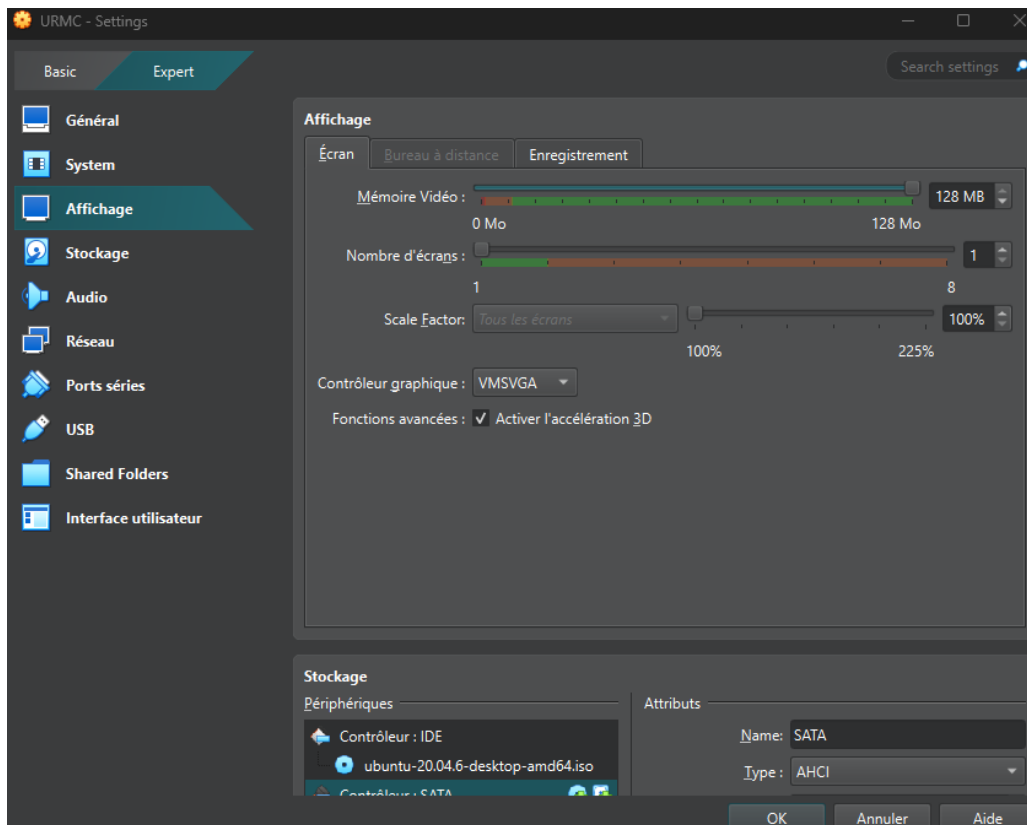


FIGURE A.4 – Accélération graphique

A.3 Configuration de la machine virtuelle

1. Sélectionner la machine virtuelle dans la liste et cliquer sur "Démarrer".
2. Suivre les étapes d'installation d'Ubuntu, en choisissant les paramètres appropriés.
3. Une fois l'installation terminée, se connecter avec les identifiants créés précédemment.

A.4 Configuration du clavier et de la résolution

Modification du clavier

- Aller dans les paramètres système, dans la section "Langues et région".
- Cliquer sur "Gérer les langues installées" et ajouter le français comme langue.
- Sélectionner le français comme langue par défaut.

Modification de la résolution

- Dans les paramètres de la machine virtuelle, aller dans l'onglet "Écran".
- Ajuster la "Définition" en fonction de la résolution de l'écran.

B Installation de Visual Studio Code

Ouvrir un terminal et exécuter les commandes suivantes :

```
sudo apt update  
sudo apt install curl apt-transport-https
```

Ajouter la clé GPG de Microsoft et le dépôt pour installer Visual Studio Code :

```
curl -sSL https://packages.microsoft.com/keys/microsoft.asc | sudo gpg  
↪ --dearmor -o /usr/share/keyrings/packages.microsoft.gpg
```

Puis, ajouter le dépôt à la liste des sources du système :

```
echo "deb [arch=amd64 signed-by=/usr/share/keyrings/packages.microsoft.  
↪ gpg] https://packages.microsoft.com/repos/code stable main" | sudo  
↪ tee /etc/apt/sources.list.d/vscode.list > /dev/null
```

Mettre à jour les sources des paquets et installer Visual Studio Code :

```
sudo apt update  
sudo apt install code
```

Si l'installation s'est bien déroulée, Visual Studio Code est disponible dans *Show Applications* en bas à gauche sur Ubuntu. Il est recommandé de faire un clic droit sur l'application et de sélectionner *Ajouter aux favoris* pour y accéder facilement.

Il est conseillé d'installer l'extension Python pour Visual Studio Code afin de bénéficier d'une prise en charge complète du langage, incluant l'auto-complétion et le débogage.